

DOKUMENTACIJA

PROJEKAT 1. ISO

Haris Vikalo

1. Opis aplikacije

Aplikacija pod nazivom "**Office Expense Manager**" služi za evidentiranje i analizu kancelarijskih troškova. Omogućava korisnicima (administratorima i zaposlenicima) unos faktura, pregled analitike po kategorijama, upravljanje budžetima i vizuelni izvještaj o potrošnji.

Tehnologije korištene u izradi:

- **Pozadinski dio (Backend):** Za razvoj glavne logike aplikacije korišten je programski jezik **Python (verzija 3.11) Flask**. Ovaj dio aplikacije obrađuje sve zahtjeve korisnika i upravlja podacima.
- **Baza podataka:** Sve informacije o korisnicima, računima i troškovima čuvaju se u **PostgreSQL 17** bazi podataka. Korištenjem Docker volumena osigurano je da podaci ostanu sačuvani i nakon što se kontejneri ugase ili obrišu.
- **Web server: Nginx** je postavljen kao "ulazna kapija" aplikacije (reverse proxy). On prima zahtjeve sa interneta i proslijeđuje ih do Flask aplikacije, što doprinosi boljoj stabilnosti i sigurnosti sistema.
- **Upravljanje kontejnerima (Orkestracija):** Cijeli sistem je organizovan pomoću alata **Docker** i **Docker Compose**. Oni omogućavaju da se svi dijelovi aplikacije (server, baza, kod) upakuju u zasebne kontejnere koji savršeno sarađuju, bez obzira na to na kojem se računaru pokreću.

2. Link repozitorija

- **URL:** https://github.com/harisoona/Office_expense_tracker_Dockerized_ISO_Project1

3. Softverski preduslovi

Za pokretanje aplikacije potrebno je imati instalirano:

- **Operativni sistem:** Windows 10/11 (sa WSL2), Linux ili macOS.
- **Docker Desktop:** Verzija 4.x ili novija.
- **Docker Compose:** Verzija 2.x ili novija.
- **Terminal:** Git Bash (MINGW64) – obavezno za pokretanje .sh skripti na Windowsu.

4. Arhitektura aplikacije

Aplikacija je zasnovana na troslojnoj arhitekturi (3-tier architecture):

1. **Klijentski sloj:** Korisnik pristupa aplikaciji preko web preglednika na portu 80.
2. **Sloj web servera (Nginx):** Prima zahtjeve i prosljeđuje ih Flask aplikaciji.
3. **Aplikacijski sloj (Flask):** Obrađuje logiku i komunicira sa bazom.
4. **Sloj baze podataka (PostgreSQL):** Skladišti podatke o korisnicima, fakturama i budžetima.

Mrežna komunikacija: Svi servisi su izolovani unutar interne Docker mreže rtpn_network.

Pohrana: Podaci baze se čuvaju u eksternom volumenu postgres_data.

5. Opis servisa, mreža i volumena

- **Servisi:**
 - rtpn_nginx: Sluša na portu 80. Konfigurisan preko nginx.conf da prosljeđuje saobraćaj na Flask servis.
 - rtpn_flask: Pokreće Python 3.11 okruženje. Sadrži aplikacijsku logiku i komunicira sa bazom koristeći environment varijable.
 - rtpn_postgres: Koristi zvanični PostgreSQL 17 image. Inicijalizuje se sa bazom office_expense_db.
- **Mreža:**
 - rtpn_network: Bridge mreža koja omogućava servisima da komuniciraju koristeći svoja imena (npr. Flask se povezuje na host postgres).
- **Volumeni:**
 - postgres_data: Trajna pohrana (Persistent Storage) koja osigurava da se podaci očuvaju iako se kontejner obriše.

6. Upute za pokretanje

Aplikacija je u potpunosti automatizovana pomoću Bash skripti. Pratite sljedeće korake:

1. **Raspakujete projektni folder.**
2. **Otvorite Git Bash** (ili drugi Bash terminal) u tom folderu.
3. **Pripremite aplikaciju pomoću skripte** (kreiranje mreža, volumena i buildanje imagea):
./pripremi_aplikaciju.sh
4. **Pokrenite aplikaciju pomoću skripte** (podizanje servisa, uvoz baze, popravka sheme i reset lozinke):
./pokreni_aplikaciju.sh
Napomena: Skripta će automatski uvesti backup.sql i postaviti šifru administratora.

7. Način pristupa aplikaciji

Aplikacija je dostupna putem web preglednika na linku <http://localhost> odnosno na portu 80.

Podaci za prijavu (Inicijalni administrator):

- **Email:** haris.vikalo@fet.ba
- **Lozinka:** admin123

8. Korišteni AI

AI sam koristio da provjerim da li mogu dockerizovati svoju aplikaciju, za ispravku grešaka koje su nastale zbog loše postavljene lokalne baze iz koje učitavam podatke, i za skraćivanje vremena u toku pravljenja dokumentacije.